SFT (Supervised Fine-Tuning) 프로세스 분석

1. SFT 개요

SFT(Supervised Fine-Tuning)는 Open R1 프로젝트의 첫 번째 핵심 단계로, 사전 훈련된 언어 모델을 추론 능력이 포함된 데이터셋으로 지도학습하는 과정입니다.

1.1 목적

- **추론 능력 학습**: `<think>...</think>` 태그 내에서 단계별 추론 과정 학습
- **형식 준수**: 정해진 대화 형식과 답변 구조 학습
- **도메인 특화**: 수학, 코딩, 과학 문제 해결 능력 향상

1.2 입력과 출력

- **입력**: 사전 훈련된 기본 모델 (예: Qwen2.5-Math-7B)
- **훈련 데이터**: 추론 트레이스가 포함된 대화 데이터셋
- **출력**: 추론 능력을 갖춘 fine-tuned 모델

2. 핵심 구현 (`src/open_r1/sft.py`)

2.1 메인 함수 구조

```
def main(script_args, training_args, model_args):
# 1. 환경 설정
set_seed(training_args.seed)
```

```
setup_logging()

# 2. 체크포인트 복구

last_checkpoint = get_last_checkpoint(training_args.output_dir)

# 3. 데이터, 토크나이저, 모델 로드

dataset = get_dataset(script_args)

tokenizer = get_tokenizer(model_args, training_args)

model = get_model(model_args, training_args)

# 4. 채팅 템플릿 설정

if tokenizer.chat_template is None:

model, tokenizer = setup_chat_format(model, tokenizer, format="chatml")

# 5. SFT 트레이너 초기화 및 훈련

trainer = SFTTrainer(...)

trainer.train()
```

2.2 데이터 로딩 시스템

단일 데이터셋 로딩

```
# 기본 사용법

dataset = datasets.load_dataset(

"open-r1/Mixture-of-Thoughts",

"all"

)
```

데이터셋 믹스처 (Dataset Mixture)

```yaml
# YAML 설정 예시
dataset_mixture:
datasets:
- id: open-r1/OpenR1-Math-220k
config: default
split: train
weight: 0.6
- id: open-r1/codeforces-cots
config: default
split: train
weight: 0.4
seed: 42
test_split_size: 0.1
```

2.3 토크나이저 설정

채팅 템플릿 처리

```python
# ChatML 형식 (기본값)
if tokenizer.chat_template is None:
model, tokenizer = setup_chat_format(model, tokenizer, format="chatml")
```

```
# Qwen 모델의 경우 EOS 토큰 설정 필요
--eos_token '<|im_end|>'


# 커스텀 채팅 템플릿 (Llama 등)
--chat_template "$(cat llama_chat_template.jinja)"
--eos_token '<|eot_id|>'
```

주요 채팅 템플릿 형식

1. **ChatML** (기본값)

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
문제를 해결해주세요.<|im_end|>
<|im_start|>assistant
<think>
추론 과정...
</think>
<answer>
최종 답변
</answer><|im_end|>
```

2. **Qwen 형식**

```
<|im_start|>user

문제 내용<|im_end|>

<|im_start|>assistant

<think>추론</think>

<answer>답변</answer><|im_end|>
```

3. 훈련 설정 및 최적화

3.1 하드웨어 설정 (8 x H100 기준)

```
# DeepSpeed ZeRO-3 사용

accelerate launch --config_file=recipes/accelerate_configs/zero3.yaml \

src/open_r1/sft.py \

--model_name_or_path open-r1/Qwen2.5-Math-7B-RoPE-300k \

--dataset_name open-r1/Mixture-of-Thoughts \

--dataset_config all \

--eos_token '<|im_end|>' \

--learning_rate 4.0e-5 \

--num_train_epochs 5 \

--max_seq_length 32768 \

--per_device_train_batch_size 2 \

--gradient_checkpointing \

--bf16 \
```

```
--use_liger_kernel \
--output_dir data/OpenR1-Distill-7B
```

3.2 주요 하이퍼파라미터

파라미터	값	설명
`learning_rate`	4.0e-5	학습률
`num_train_epochs`	5	훈련 에포크 수
`max_seq_length`	32768	최대 시퀀스 길이
`per_device_train_batch_size`	2	디바이스당 배치 크기
`gradient_checkpointing`	True	메모리 절약을 위한 그래디언트 체크포인팅
`bf16`	True	BFloat16 혼합 정밀도
`use_liger_kernel`	True	메모리 효율적인 커널 사용

3.3 메모리 최적화 기법

DeepSpeed ZeRO 설정

```yaml
# recipes/accelerate_configs/zero3.yaml
compute_environment: LOCAL_MACHINE
deepspeed_config:
gradient_accumulation_steps: 1
gradient_clipping: 1.0
offload_optimizer_device: cpu
```

```
offload_param_device: cpu

zero3_init_flag: true

zero3_save_16bit_model: true

zero_stage: 3
```

Liger Kernel 최적화

- **메모리 사용량 감소**: 표준 구현 대비 20-30% 메모리 절약
- **속도 향상**: 최적화된 CUDA 커널 사용
- **호환성**: Transformers 라이브러리와 완전 호환

4. 데이터 전처리

4.1 대화 형식 변환

```
def  format_conversation(example):

"""대화 형식으로 데이터 변환"""

messages = []

# 시스템 프롬프트 (선택적)

if system_prompt:

messages.append({"role": "system", "content": system_prompt})

# 사용자 질문

messages.append({"role": "user", "content": example["prompt"]})

# 어시스턴트 응답 (추론 + 답변)

messages.append({"role": "assistant", "content": example["response"]})
```

```
return {"messages": messages}
```

4.2 토큰화 과정

```
def tokenize_function(examples):
"""채팅 템플릿을 적용하여 토큰화"""
tokenized = tokenizer.apply_chat_template(
examples["messages"],
tokenize=True,
add_generation_prompt=False,
return_tensors="pt",
max_length=max_seq_length,
truncation=True
)
return {"input_ids": tokenized}
```

5. 훈련 모니터링

5.1 Weights & Biases 통합

```
# 자동 로깅 설정
if "wandb" in training_args.report_to:
```

```
init_wandb_training(training_args)
```

주요 메트릭

- **Loss**: 훈련 손실 및 검증 손실
- **Learning Rate**: 학습률 스케줄링
- **GPU Memory**: 메모리 사용량 모니터링
- **Throughput**: 초당 처리 토큰 수

5.2 체크포인트 관리

```
# 자동 체크포인트 저장
training_args.save_steps = 500
training_args.save_total_limit = 3

# 체크포인트에서 복구
last_checkpoint = get_last_checkpoint(training_args.output_dir)
if last_checkpoint:
trainer.train(resume_from_checkpoint=last_checkpoint)
```

6. 실제 사용 예시

6.1 OpenR1-Distill-7B 훈련

```
# 1. 환경 설정
```

```
export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export WANDB_PROJECT=open-r1

# 2. 훈련 실행
ACCELERATE_LOG_LEVEL=info accelerate launch \
--config_file recipes/accelerate_configs/zero3.yaml \
src/open_r1/sft.py \
--config recipes/OpenR1-Distill-7B/sft/config_distill.yaml
```

6.2 작은 모델 실험 (1.5B)

```
accelerate launch --config_file recipes/accelerate_configs/zero3.yaml \
src/open_r1/sft.py \
--model_name_or_path Qwen/Qwen2.5-1.5B \
--eos_token '<|im_end|>' \
--dataset_name open-r1/Mixture-of-Thoughts \
--dataset_config all \
--learning_rate 4.0e-5 \
--num_train_epochs 1 \
--max_seq_length 32768 \
--per_device_train_batch_size 16 \
--gradient_checkpointing \
--bf16 \
```

```
--use_liger_kernel \
--output_dir data/Qwen2.5-1.5B-Open-R1-Distill
```

7. 성능 최적화 팁

7.1 배치 크기 조정

```
# 글로벌 배치 크기 = per_device_batch_size × num_gpus × gradient_accumulation_step
# 메모리에 맞춰 per_device_batch_size 조정
# GPU 수 변경 시 배치 크기도 비례하여 조정
```

7.2 시퀀스 길이 최적화

```
# 긴 시퀀스 처리를 위한 설정
--max_seq_length 32768  # 추론 트레이스를 위해 긴 컨텍스트 필요
--gradient_checkpointing # 메모리 절약
```

7.3 멀티노드 확장

```
# Slurm을 통한 멀티노드 훈련
sbatch  --nodes=4  slurm/train.slurm \
--model OpenR1-Distill-7B \
--task  sft \
```

```
--config distill \
--accelerator zero3
```

8. 문제 해결

8.1 일반적인 오류

CUDA 메모리 부족

```
# 해결 방법
--per_device_train_batch_size 1 # 배치 크기 감소
--gradient_checkpointing # 메모리 절약
--max_seq_length 16384 # 시퀀스 길이 감소
```

토크나이저 오류

```
# Qwen 모델의 경우 반드시 EOS 토큰 설정
--eos_token '<|im_end|>'

# 커스텀 템플릿 사용 시 템플릿과 EOS 토큰 일치 확인
--chat_template "$(cat template.jinja)"
--eos_token '<|eot_id|>'
```

8.2 성능 최적화

느린 훈련 속도

```
# 해결 방법

--use_liger_kernel  # 최적화된 커널 사용

--bf16  # 혼합 정밀도

--dataloader_num_workers  4  # 데이터 로딩 병렬화
```

SFT 프로세스는 Open R1의 기초가 되는 중요한 단계로, 올바른 설정과 최적화를 통해 DeepSeek-R1과 동등한 성능의 모델을 훈련할 수 있습니다.